

Test Plan

Proyecto 1 — Ambiente de Verificación en Capas para Bus con Árbitro

Integrantes: Gerald Balnco Sáenz y Leonardo Pérez Sandoval

Fecha de entrega: jueves 1 de octubre de 2026

1. Objetivo

Definir la estrategia de verificación funcional del DUT (bus con árbitro y FIFOs), especificando la arquitectura del ambiente de pruebas, el formato de los paquetes de comunicación entre los bloques del ambiente, y las funcionalidades del diseño que serán cubiertas mediante pruebas con aleatorización controlada (constrained-random), incluyendo los casos de esquina relevantes.

2. Descripción del DUT

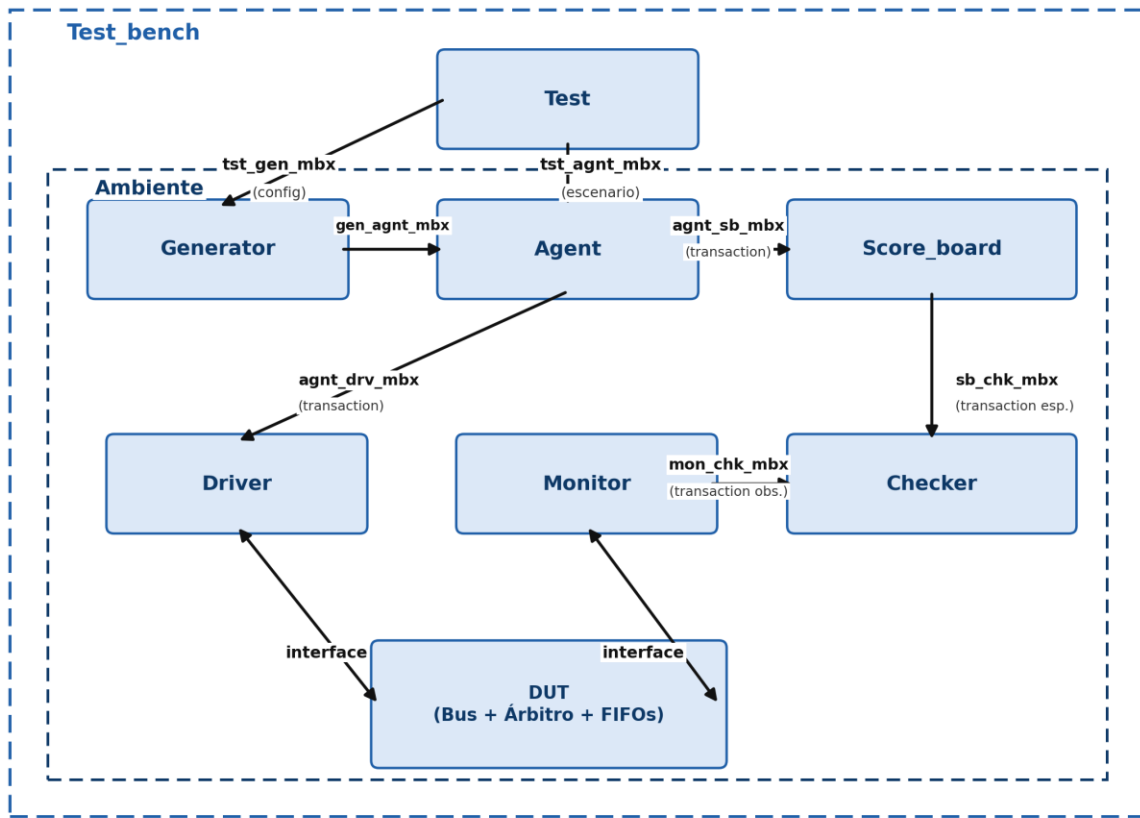
El DUT es un bus compartido al que se conectan M dispositivos, cada uno con su propio FIFO de entrada y salida (TX y RX) y su capa de protocolo. El acceso al bus es gestionado por un árbitro que administra las señales de push/pop y pending de cada FIFO, controla el intercambio de señales de transmisión y habilita el paso de datos entre los serializadores (SRLZ) de entrada y salida del bus.

2.1 Interfaz del DUT (basado en el prototipo)

Parámetro	Valor	Descripción
bits	1	Ancho de las señales de control por dispositivo (no modificable).
drvrs	4	Número de dispositivos (drivers) conectados al bus (modificable).
pckg_sz	16	Ancho en bits del paquete de datos. Debe soportar 16, 32 y 64 (modificable).
broadcast	{8{1'b1}}	Identificador reservado de dirección de destino que representa un mensaje de difusión a todos los dispositivos (preferiblemente no modificable).

3. Arquitectura del ambiente de verificación

El ambiente se implementa como un testbench en capas (signal, command, functional, escenario y test). Por indicación del profesor, para este proyecto no se implementan las capas de Assertions ni de Functional Coverage; la verificación de correctitud recae completamente en el Scoreboard y el Checker.



3.1 Función de cada componente

Componente	Función
Test	Define las restricciones de aleatoriedad para cada escenario (rangos de datos, configuración del dispositivo/ambiente, excepciones de protocolo, tipos y secuencia de errores, retardos y sincronizaciones).
Generator	Genera la secuencia de funciones o transacciones que conforman un escenario de prueba
Agent	Traduce instrucciones complejas en comandos simples para el Driver; envía la transacción también al Scoreboard.
Scoreboard	Almacena las transacciones esperadas para que el Checker las compare contra lo observado en el DUT.
Checker	Compara lo recibido por el Monitor contra el Scoreboard y reporta discrepancias (paquete corrupto, retraso fuera de rango, destino incorrecto, etc.).
Driver	Convierte los comandos abstractos del Agent en las señales de entrada del DUT para cada dispositivo.
Monitor	Observa las señales de salida del DUT y las traduce en eventos o transacciones para el Checker.
DUT	Bus con árbitro y FIFOs TX y RX.

4. Formato de los paquetes de comunicación

Se define un paquete lógico de transacción (a nivel del ambiente, por encima de las señales D_push/D_pop del RTL) con los siguientes campos:

Señal	Ancho / Tipo	Descripción
id_transaccion	entero	Identificador único incremental de la transacción, usado para trazabilidad en el reporte CSV.
d_origen	log2(drvers) bits	Dispositivo que origina el mensaje.
d_destino	log2(drvers) bits broadcast	Dispositivo destino, una dirección inexistente (para forzar mensajes con error) o el identificador de broadcast.
longitud	16 32 64	Tamaño del paquete de datos, aleatorizado entre los tres valores soportados.
payload	longitud bits	Datos aleatorizados del paquete.
error	booleano	Indica si el paquete se dirige intencionalmente a un dispositivo inexistente.
t_envio	tiempo (ns)	Marca de tiempo en que el Driver inyecta el paquete al DUT.
t_recibido	tiempo (ns)	Marca de tiempo en que el Monitor observa el paquete en el destino.

5. Plan de pruebas General

Objetivo: Es la prueba principal del ambiente. Su propósito es estresar el DUT de manera integral permitiendo que el Generador y Test resuelvan todas las variables al azar de manera simultánea. En teoría, al ejecutar este escenario con infinitas semillas de aleatorización, se lograría una cobertura absoluta del estado del diseño.

Qué se aleatoriza: Se habilita tráfico concurrente en los M dispositivos, aleatorizando dinámicamente en cada transacción: número de transacciones, las direcciones destino (unicast y broadcast), los tiempos de retardo entre envíos y la inyección ocasional de direcciones erróneas.

Manejo del Largo del Paquete (Compile-time): Dado que el ancho del paquete (pkg_sz) es un parámetro estructural del hardware, este no se aleatoriza en tiempo de corrida. El Caso General se ejecutará en tres iteraciones de compilación independientes (configurando el DUT y el Ambiente para 16, 32 y 64 bits respectivamente), resolviendo así el requerimiento de cobertura para todos los anchos soportados.

Cómo se verifica: El Monitor capta todo el tráfico cruzado, el Scoreboard enruta las transacciones esperadas y el Checker valida que ningún paquete se pierda, se corrompa o se entregue al dispositivo equivocado bajo condiciones de tráfico puramente aleatorias.

6. Casos de esquina identificados

- **Todos los dispositivos solicitando el bus en el mismo ciclo.** Se varía el número de solicitantes simultáneos (de 2 hasta M); expone bugs de prioridad fija o starvation en el árbitro que con tráfico normal podrían no manifestarse nunca.
- **FIFO de un dispositivo llena mientras otro sigue transmitiendo.** Se varía la tasa de llegada de transacciones a ese terminal para forzar la saturación mientras el resto opera normal
- **Mensajes consecutivos con error (destino inexistente) desde el mismo terminal.** Se varía la longitud de la ráfaga de errores seguidos (3, 5, 10); confirma que el manejo de errores no acumula estado ni degrada el desempeño del bus con el tiempo.
- **Transacción con el mínimo y el máximo retardo permitido entre mensajes.** Se fija el retardo exactamente en los límites del rango (no aleatorio dentro de él); los bugs de temporización casi siempre aparecen en los bordes, no en el promedio, y el random puro rara vez los toca.
- **FIFO de un dispositivo vacía y se solicita pop a pesar de ello.** Se varía el momento exacto de la solicitud respecto al último dato disponible; si no se maneja el underflow, el diseño podría entregar datos basura como si fueran válidos, generando falsos positivos silenciosos.
- **Patrón de datos alternante (checkerboard) en el payload.** Se varía el ancho del patrón según la longitud del paquete (16/32/64) y cuántas transacciones consecutivas lo mantienen; detecta acoplamiento entre bits adyacentes que pasa desapercibido con datos puramente aleatorios, sobre todo al pasar por los serializadores.
- **Push y pop simultáneos en la misma FIFO.** Se varía si la FIFO está vacía, parcial o llena al momento de forzar la operación; es un caso clásico de condición de carrera en FIFOs que puede perder o duplicar datos sin que sea evidente en pruebas simples.
- **Múltiples dispositivos operando simultáneamente (push y pop concurrentes).** Se varía cuántos dispositivos actúan a la vez y si sus operaciones son del mismo tipo o mixtas; prueba que el árbitro y las FIFOs manejan la concurrencia real del sistema, no solo actividad de a un dispositivo por vez.
- **Patrón de actividad alternante entre dispositivos.** Se varía el período de alternancia y qué par de dispositivos se alterna; es un patrón dirigido que el random puro difícilmente genera y expone problemas de sincronización que solo aparecen con cambios de actividad repetitivos y predecibles.
- **FIFO llena con pops consecutivos.** Se llena la FIFO por completo y se mantienen solicitudes de pop consecutivas. Se varía la velocidad de los pops respecto a la de los pushes.

7. Análisis de resultados

El ambiente debe generar, al final de cada corrida:

- Un archivo CSV con: id_transaccion, t_envio, d_origen, d_destino, t_recibido, retraso.
Un histograma de los retardos de los paquetes recibidos, a partir del CSV anterior.

8. Ejecución concurrente

Cada unidad del ambiente (Test, Generator, Agent/Driver por dispositivo, Monitor por dispositivo, Scoreboard y Checker) se ejecutan en un proceso independientes, sincronizados mediante los eventos y mailboxes/queues correspondientes.